Part 2: Testing Analysis – Viola Zeneli

Boundary Value Testing

Method: DefaultCirculationService.calculateLateFeeWithDiscount(String itemCode, int discountPercent)

Boundary Value Testing is used to verify the behavior of the method at the **limits of valid input values**, where errors are most likely to occur.

In this method, BVT was applied only to **discountPercent** because it is the only input parameter with explicitly defined boundaries **[0,100]**.

I chose **Robust BVT**, because it checks **both valid and invalid boundaries**, so test cases cover values (**-1, 0, 1, 50, 99, 100, 101**).

TC	itemCode	discountPercent	Boundary Tested	Expected Result
BVT_1	"BOOK-001"	-1	Below min discount	IllegalArgumentException
BVT_2	"BOOK-001"	0	Min discount	Full fine returned
BVT_3	"BOOK-001"	1	Just above min	Fine reduced by 1%
BVT_4	"BOOK-001"	50	Nominal value	Fine reduced by 50%
BVT_5	"BOOK-001"	99	Just below max	Fine reduced by 99%
BVT_6	"BOOK-001"	100	Max discount	Late fee = 0
BVT_7	"BOOK-001"	101	Above max discount	IllegalArgumentException

Equivalence Class Testing

Method: DefaultCirculationService.assessItemReturnStatus(String itemCode, int daysLate)

Equivalence Class Testing is used to group input values that are expected to produce the **same output**, so only one representative value from each group needs to be tested.

For this method, equivalence classes are identified based on: the validity of itemCode, whether an active borrow exists and the range of daysLate.

Weak Robust Equivalence Class Testing was applied by selecting one representative input from each valid and invalid equivalence class without testing combinations of classes.

TC	itemCode	daysLate	Equivalence Class	Expected Result
EC_1	null	2	Invalid itemCode	IllegalArgumentException
EC_2	"BOOK-404"	2	No active borrow exists	"NO_ACTIVE_BORROW"
EC_3	"BOOK-001"	2	Already returned item	"ALREADY_RETURNED"
EC_4	"BOOK-001"	-1	Early return (daysLate<0)	"EARLY_RETURN"
EC_5	"BOOK-001"	0	On-time return (daysLate=0)	"ON_TIME"
EC_6	"BOOK-001"	2	Slightly late (1<=daysLate<=3)	"SLIGHTLY_LATE"
EC_7	"BOOK-001"	5	Moderately late (4<=daysLate<=7)	"MODERATELY_LATE"
EC_8	"BOOK-001"	10	Very late (8<=daysLate<=14)	"VERY_LATE"
EC_9	"BOOK-001"	20	Severely overdue (daysLate>14)	"SEVERELY_OVERDUE"

Code Coverage Testing

Method: DefaultCirculationService.canRenewBorrow(int memberId, String itemCode)

1. Statement Coverage

Statement coverage requires that every executable statement in the method is executed at least once, regardless of the outcome of decision points.

TC	memberId	itemCode	Path (where execution ends)	Expected
SC_1	-1	"BOOK-001"	Ends at **first if** (input validation)	false
SC_2	1	"BOOK-001"	Ends at **second if** (null check)	false
SC_3	1	"BOOK-001"	Ends at **final return**	true

2. Branch Coverage

Branch coverage requires that each decision (if) has both TRUE and FALSE outcomes executed at least once. It is about decision points, not individual boolean conditions.

TC	memberId	itemCode	Path (branches taken)	Expected
BC_1	-1	"BOOK-001"	First if = **true**	false
BC_2	1	"BOOK-001"	First if = **false**, Second if = **true**	false
BC_3	1	"BOOK-001"	First if = **false**, Second if = **false**	true

Naming Table of Boolean Conditions Used in Decision Logic of this Method

These condition names are used to avoid repeating full boolean expressions when defining condition coverage and MC/DC test cases later.

ID	Boolean Condition	Location in Code
C1	memberId <= 0	First if (input validation)
C2	itemCode == null	First if (input validation)
C3	itemCode.trim().isEmpty()	First if (input validation)
C4	member == null	Second if (existence check)
C5	item == null	Second if (existence check)
C6	record == null	Second if (existence check)
C7	record.getMemberId() == memberId	Final return (authorization check)
C8	!record.isReturned()	Final return (status check)
C9	!record.isOverdue(today)	Final return (time check)
C10	member.isActive()	Final return (account check)
C11	!borrowStorage.hasOverdueBorrows (memberId, today)	Final return (policy check)

3. Condition Coverage (Basic / Weak)

Condition coverage requires that each individual boolean condition in a decision evaluates to both TRUE and FALSE at least once, regardless of the overall decision outcome.

TC	memberId	itemCode	Forced Condition	Condition Value	Other Conditions	Expected Result
CC_1	-1	"BOOK-001"	C1	TRUE	others irrelevant	false
CC_2	1	null	C2	TRUE	others irrelevant	false
CC_3	1	" "	C3	TRUE	others irrelevant	false
CC_4	1	"BOOK-001"	C4	TRUE	C5,C6=FALSE	false
CC_5	1	"BOOK-001"	C5	TRUE	C4,C6=FALSE	false
CC_6	1	"BOOK-001"	C6	TRUE	C4,C5=FALSE	false
CC_7	1	"BOOK-001"	C7	FALSE	others TRUE	false
CC_8	1	"BOOK-001"	C8	FALSE	others TRUE	false
CC_9	1	"BOOK-001"	C9	FALSE	others TRUE	false
CC_10	1	"BOOK-001"	C10	FALSE	others TRUE	false
CC_11	1	"BOOK-001"	C11	FALSE	others TRUE	false
CC_12	1	"BOOK-001"	none	all TRUE	true	true

4. MC/DC Coverage

MC/DC (Modified Condition/Decision Coverage) requires that each condition is shown to independently affect the decision outcome while all other conditions are held constant.

MC/DC Scope (All decisions of this method)

Decision ID	Code Location	Decision Expression
D1	First if (input validation)	if (C1 || C2 || C3)
D2	Second if (existence check)	if (C4 || C5 || C6)
D3	Final return decision	C7 && C8 && C9 && C10 && C11

MC/DC Test Cases for D1, D2, D3 respectively

TC	C1	C2	C3	Decision Outcome	Return
D1_M1	F	F	F	FALSE	?
D1_M2	T	F	F	TRUE	FALSE
D1_M3	F	T	F	TRUE	FALSE
D1_M4	F	F	T	TRUE	FALSE

TC	C4	C5	C6	Decision Outcome	Return
D2_M1	F	F	F	FALSE	?
D2_M2	T	F	F	TRUE	FALSE
D2_M3	F	T	F	TRUE	FALSE
D2_M4	F	F	T	TRUE	FALSE

TC	C7	C8	C9	C10	C11	Decision Outcome	Return
D3_M1	T	T	T	T	T	TRUE	TRUE
D3_M2	F	T	T	T	T	FALSE	FALSE
D3_M3	T	F	T	T	T	FALSE	FALSE
D3_M4	T	T	F	T	T	FALSE	FALSE
D3_M5	T	T	T	F	T	FALSE	FALSE
D3_M6	T	T	T	T	F	FALSE	FALSE

#NOTE: If you go to the next table, the previous tables conditions are all false.